Mecanismos de ejecución de código en el servidor y en un navegador web.

Como ya es conocido, la web está basada en la distribución de la información accesible a través de una conexión a Internet mediante el empleo de **hipertexto** . En sus forma más básica disponemos de un **servidor web** capaz de enviar información (texto, video y multimedia) a unos **clientes web** que no son más que equipos que disponen de una aplicación (**navegador web**) capaz de interpretar la información recibida.

Tanto la comunicación como el envío de información se basa en el uso de **estándares** ampliamente aceptados (en origen, **HTTP** para la comunicación entre el servidor y el cliente, **HTML** para el formateo de la información y **URI** para la localización de los recursos disponibles).

En este contexto, pronto se vió la necesidad de añadir capacidad de proceso y de toma de decisiones que permitieran convertirla en una plataforma para el desarrollo de aplicaciones complejas. Y, ante esta necesidad, surgieron dos modos de trabajo diferentes:

- Creación de **aplicaciones del lado servidor** : en este caso, existe un **servidor de aplicaciones** (que puede o no ser el **servidor web**) que, ante determinadas peticiones del usuario (basadas en una URL específica o que tengan una extensión predeterminada), **ejecutan código** que es capaz de generar recursos (páginas **HTML**, **XHTML**, **XML**, ficheros binarios, etc) que se envían al cliente para su procesamiento. De este modo es posible que, ante **diferentes peticiones del usuario** (normalmente mediante el uso de formularios o enlaces) el servidor **retorne diferentes recursos**. Actualmente hay una amplia gama de herramientas que permiten desarrollar aplicaciones del lado servidor como **Python, JAVA/J2EE**, **NET**, **PHP**, **Javascript** etc.
- Ejecución de **código en el lado cliente** (en el navegador web) : a la par que se iban desarrollando herramientas para la creación de código del lado servidor, se iba haciendo evidente que esta tecnología tenía una desventaja importante : la necesidad de **enviar** la página al servidor cada vez que se precisa un procesamiento de la misma. Esto conllevaba un tiempo inadmisible para determinadas tareas simples como **validación de datos** o **completado de información** o incluso para aplicaciones que no requieren de un procesamiento en el lado del servidor (como aplicaciones de conversión de datos, juegos, etc.). El lenguaje más utilizado actualmente para el desarrollo de código en el lado cliente es **Javascript** (puede utilizarse directamente o a través de frameworks como jQuery, Angular, React…).

Como resumen podríamos decir que el código que se ejcuta en el lado **cliente** es ejecutado por el **navegador web** del usuario (puede que con el uso de plugins o aplicaciones adicionales), mientras que el código del lado **servidor** es ejecutado por un **servidor de aplicaciones** en un equipo remoto.

Diferencias

Como ya se ha visto, la principal diferencia entre la ejecución de código en el lado cliente y el lado servidor radica, precisamente, en dónde se ejecuta el código. Sin embargo, hay otras diferencias que conviene tener en cuenta :

- La ejecución del lado **cliente** es **más rápida** que la ejecución del lado del **servidor**, puesto que no requiere el envío de información al servidor
- En general, las aplicaciones del lado del **servidor** son más seguras, dado que en el **cliente** el código está en texto plano y puede ser accesible por cualquiera
- En cuanto al **funcionamiento**, hay que tener claro que el código de scripting del lado cliente puede ser **bloqueado por el usuario** mediante las opciones del navegador, con lo cual no llegaría a ejecutarse
- La ejecución del lado **servidor no tiene restricciones en el acceso a los datos** : puede conectar con servidores de bases de datos y de aplicaciones, guardar información en el servidor, etc. Muchas de estas tareas suelen estar limitadas en las aplicaciones del lado cliente por razones de seguridad

Tradicionalmente lo habitual ha sido **combinar ambas soluciones** en el caso de aplicaciones empresariales. El código propiamente dicho de la aplicación (control de acceso, acceso a bases de datos y renderización de páginas y control de la sesión del usuario) se llevaría a cabo en el **servidor**, mientras que tareas de validación simple y modificación dinámica del aspecto de la página se llevarían a cabo en el lado del **cliente**.

Por otro lado, la aparición de **AJAX** (**Asynchronous JavaSctript And XML**) está modificando en gran medida el uso del código del lado cliente. Mediante AJAX se pueden diseñar aplicaciones web **más interactivas** que, incluso, puedan simular a las aplicaciones de escritorio tradicionales.

En este sentido, los framewoks como Angular, React, etc han extendido el diseño de aplicaciones complejas del lado cliente (mediante la SPA - Single Page Application -) que son aplicaciones que, aunque se ejecutan en una única página permiten el uso de rutas y simulan una aplicación compleja del lado servidor. Estas aplicaciones se combinan con servicios de acceso a datos remotos basados en tecnología de servidor (habitualmente a través de servicios REST) para diseñar aplicaciones web tan robustas y seguras como aquellas definidas del lado servidor.

En la práctica podemos combinar estas opciones entre sí:

- un servicio REST que retorna datos en base a las peticiones de los clientes (empleando los mecanismos de seguridad adecuados)
- una aplicación cliente que accede a dicha información
- una aplicación servidora que accede también a dicha información

Lenguajes de Marcas

En el entorno de desarrollo de aplicaciones web es fundamental el uso de lenguajes de marcas que permiten enviar **metainformación** junto con la información propiamente dicha. Esta metainformación se implementa mediante etiquetas y puede referirse a la **presentación** de la información (**HTML, XML, CSS**…), o a la **estructura del texto** (**XML**).

JavaScript

JavaScript es un **lenguaje interpretado** que se ejecuta en el navegador del usuario y que fue desarrollado por Brendan Eich de Netscape llamándose en origen **Mocha**, luego **LiveScript** y, por último, **JavaScript** (que fue integrado en Netscape 2.0 a finales de 1995).

Es un lenguaje **orientado a objetos** (permite definir y usar objetos), **débilmente tipado** (no es necesario declarar el tipo de las variables) y **basado en prototipos** (aunque permite el uso de objetos hay diferencias fundamentales con los lenguajes **basados en clases** como ya veremos posteriormente).

Dado su éxito, Microsoft sacó su propia versión (**JScript**) aunque actualmente el estándar es JavaScript (también llamado **ECMAScript - European Computer Manufacturers Association**). JScript deriva de ECMAScript, así que soporta todas sis características. Actualmente la última versión aprobada es ECMAScript 2019. Un problema con este estándar es que los navegadores no suelen ser compatibles con todas sus características (sobre todo los navegadores antiguos) lo que complica el desarrollo. Actualmente esto se intenta subsanar

mediante el uso de **transpiladores** (traducen el código actual a versiones anteriores) y **polyfills** (implementan en versiones anteriores nuevas funcionalidades del lenguaje). Los nuevos frameworks basados en JavaScript ya incluyen estas herramientas de modo que el código generado sea lo más compatible posible.

Aunque Javascript es un lenguaje débilmente tipado, existen extensiones al mismo que sí admiten el tipado fuerte. Entre estas destaca Typescript creado y mantenido por Microsoft y que es la base de Angular (framework que estudiaremos en este curso).